

4-Wire SPI Slave Interface Design

A Verilog Implementation of the SPI Protocol Register Map Access

Presented to:

Analog Devices

Eng. Fatma Ali

Presented by:

Mohamed Abdelhafez Elnewehy

July 2026

I designed and implemented a 4-wire SPI slave controller (CSB, CLK, SDI, SDO) that provides read and write access to an internal 32768 x 8 register map, following the standard SPI frame format defined in the assignment (16-bit header followed by one or more 8-bit data bytes, MSB first).

The design supports single-byte and multi-byte burst access with automatic address increment, and meets the combinational read requirement, where read data must appear on SDO within half a clock cycle after the last address bit is sampled.

For writes, the write-enable and write-data signals are generated combinatorially so that the last byte of any transaction (single write or the final byte of a burst) is committed on the same clock edge it completes, without requiring an extra clock cycle.

CSB is used as an asynchronous control signal: asserting it starts a transaction, and de-asserting it (even mid-transaction) resets the internal state, so the slave always recovers cleanly for the next transaction.

Simulation

The design was verified using a self-checking testbench with six tasks, each covering one scenario:

- 1. Single Write** — Writes one byte to a register and checks that the register map received the correct value.
- 2. Single Read** — Preloads a register with a known value and checks that the SPI slave outputs the correct byte on SDO.
- 3. Write Burst (4 bytes)** — Writes four consecutive bytes in a single CSB assertion and checks that all four registers were updated with auto-incrementing addresses.
- 4. Read Burst (4 bytes)** — Preloads four registers and reads them back in one burst, checking that all four bytes are received correctly in order.
- 5. CSB Mid-Transaction Abort** — De-asserts CSB in the middle of a transaction and checks that no spurious write occurs, then confirms the slave recovers correctly on the next clean transaction.
- 6. Address Wraparound** — Performs a burst write starting near the top of the address space (0x7FFE) and checks that the address correctly wraps from 0x7FFF back to 0x0000.

```
run 10000 ns
[PASS] Test 1: Single Write
[PASS] Test 2: Single Read
[PASS] Test 3: Write Burst (4 bytes)
[PASS] Test 4: Read Burst (4 bytes)
[PASS] Test 5: CSB Mid-Transaction Abort
[PASS] Test 6: res Wraparound (0x7FFF->0x0000)
-----
TOTAL: 6    PASS: 6    FAIL: 0
-----
```